Ranking	Vulnerability Scanner
1	OWASP ZAP
2	Nikto
3	BurpSuite | Trivy
4	Nessus
5	Nuclei | Acunetix

Ranking	Practice Test Application
1	DVWA

Vulnerability scanners:

OWASP ZAP (Open source): https://github.com/zaproxy/zaproxy

Automated scan: Defines the application to attack

Scan Policy Manager: Deals with Scan Policy (What tests ZAP will do to the application)

Alerts: Show the vulnerabilities found by ZAP

https://www.zaproxy.org/docs/api/#introduction REST API that allows you to automate scans and pull results.

Justification For OWASP ZAP:

Free, open-source DAST tool with proper documentation, and very popular. Automated Scan allows getting a baseline "before" list of findings from the target application (Juice Shop/DVWA) without configuring each scan manually. Scan policy manager controls which tests run and at what strength to keeps the scan relevant and results reproducible for a fair before/after comparison. Rest API enables automation of triggering scans and fetching results making it easier to pipe to a Python script.

Nikto (Open source): https://github.com/sullo/nikto

-h (host): Defines the application to attack.

-tuning (tuning): Deals with test categories Nikto will run.

-o (output): Shows vulnerabilities found.

Severity Ratings:

No REST API, but CLI flags allow for automation https://github.com/sullo/nikto/wiki

BURP (Proprietary):

Target > Scope: Defines the application to attack

Scanner (PRO version): Scans vulnerabilities in application

Issues/Dashboard: Shows vulnerabilities found

Severity Ratings: High/Medium/Low/Info | Confidence: Certain/Firm/Tentative

https://portswigger.net/burp/documentation/desktop/testing-workflow/working-with-graphql Use GraphQL API to automate scans and pull results.

Wapiti (Open source): https://github.com/wapiti-scanner/wapiti

-u (URL): Defines the application to attack.

-m (modules): Deals with which vulnerability modules will be tested for.

-o (output): Shows vulnerabilities found.

Severity Ratings: 4 (Critical) to 1 (Low)

No REST API, but CLI flags allow for automation https://wapiti-scanner.github.io/

NESSUS (Proprietary):

New Scan -> Scan Template: Defines the application to attack

Vulnerabilities: Show the vulnerabilities found by the scan template

Severity Ratings: Critical/High/Medium/Low/Informational

https://5c3p73r404.medium.com/how-to-interact-with-nessus-api-to-automate-the-scans-eb935fb7952a REST API that allows you to automate scans and pull results.

Nuclei (Open source): https://github.com/projectdiscovery/nuclei

Target: Defines the application to attack.

Templates: YAML-based checks defining what vulnerabilities to test for.

Output: JSON showing vulnerabilities found with matched template.

Severity Ratings: Unknown /Critical/High/Medium/Low/Informational.

No REST API, but CLI flags allow for automation
https://docs.projectdiscovery.io/opensource/nuclei/running

Acunetix (Proprietary):

Scans: Run vulnerability scan against a target.

Vulnerabilities: Show the vulnerabilities found by the scan.

Severity rating: Critical/High/Medium/Low/Informational

https://www.acunetix.com/support/docs/getting-comfortable-with-acunetix-apis/ REST API that allows you to automate scans and pull results.

Invicti (Proprietary):

New Scan: Defines the application to attack.

Vulnerabilities/Issues Panel: Shows vulnerabilities found.

Severity Ratings: Critical/High/Medium/Low

REST API https://docs.invicti.com/ip/category/platform-api

Qualys WAS (Proprietary):

Web Applications Tab: Defines the application to attack.

Scans Tab: Runs the scan.

KnowledgeBase/Detections: Shows vulnerabilities found.

Severity: 5 (Highest) to 1 (Lowest).

REST API https://docs.qualys.com/en/was/api/get_started/get_started.htm

SQLMap (Open Source): https://github.com/sqlmapproject/sqlmap

-u (URL): Defines the application to attack.

--level/--risk: Controls test depth.

Severity Ratings: None

REST API https://github.com/sqlmapproject/sqlmap/wiki/usage

Arachni: https://github.com/Arachni/arachni (No longer maintained)

WPScan (free-to-use for personal): WordPress scanner

OpenVAS (free-to-use for personal): Network vulnerability scanner

OWASP TOP 10 (2025): https://owasp.org/Top10/2025/

1. Broken Access Control E.g. Cross-site Request Forgery (CWE-352)
   No policies enforcing user's permissions.

2. Security Misconfiguration E.g. Use of Default Credentials (CWE-1392)
   System, application, or cloud is set up incorrectly from a security perspective, creating vulnerabilities

3. Software Supply Chain Failure E.g. Dependency on Vulnerable Third-Party Software. (CWE-1395)
   Compromises in the process of building, distributing, or updating software.

4. Cryptographic Failures E.g. Use of Predictable Algorithm in Random Number Generator (CWE-1241)

5. Injection E.g. Improper Neutralisation of Special Elements used in an SQL Command (CWE-89)

   Vulnerability that allows untrusted user input to be sent to a high-trust system component (e.g. browser, database), and the system executes the input.

6. Insecure Design E.g. Unrestricted Upload of File with Dangerous Type (CWE-434)
   A broad category representing different weaknesses, expressed as "missing or ineffective control design."

7. Authentication Failures E.g. Use of Hard-coded Credentials (CWE-798)
   When an attacker is able to trick a system into recognising an invalid or incorrect user as legitimate.

8. Software or Data Integrity Failures E.g. Inclusion of Functionality from Untrusted Control Sphere (CWE-829)
   Relates to code and infrastructure that does not protect against invalid or untrusted code or data being treated as trusted and valid.

9. Security Logging and Alerting Failures E.g. Insufficient logging (CWE-778)
   Without logging and monitoring, attacks and breaches cannot be detected, and without alerting it is very difficult to respond quickly and effectively during a security incident.

10. Mishandling of Exceptional Conditions E.g. Not Failing Securely (CWE-636)
    Happens when programs fail to prevent, detect, and respond to unusual and unpredictable situations, which leads to crashes, unexpected behaviour, and sometimes vulnerabilities.

Secure Design Principles

1. **Economy of mechanism**: keep the design of the system as simple and small as possible.
2. **Fail-safe defaults**: base access decisions on permission (a user is explicitly allowed access to a resource) rather than exclusion (a user is explicitly denied access to a resource).
3. **Complete mediation**: every access to every object must be checked for authorization.
4. **Least privilege**: every program and every user of the system should operate using the least set of privileges necessary to complete the job.
5. **Least common mechanism**: minimize the amount of mechanism (resources, components, or services) common to more than one user and depended on by all users.
6. **Psychological acceptability**: it is essential that the human interface be designed for ease of use, so that users routinely and automatically apply the protection mechanisms correctly.
7. **Compromise recording**: it is sometimes suggested that mechanisms that reliably record that a compromise of information has occurred can be used in place of more elaborate mechanisms that completely prevent loss.
8. **Defence in depth**: design the system so that it can resist attack even if a single security vulnerability is discovered or a single security feature is bypassed. May

involve including multiple levels of security mechanisms or designing a system so that it crashes rather than allowing an attacker to gain complete control.

9. **Fail securely**: a counterpoint to defence in depth is that a system should be designed to remain secure even if it encounters an error or crashes.
10. **Design for updating**: no system is likely to remain free from security vulnerabilities forever, so developers should plan for the safe and reliable installation of security updates